Enabling UEFI Secure Boot Across Modern Build Systems

Vasquez, Frank



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Agenda

- UEFI Secure Boot
- MOK solution for alternative Linux distros
- Bluefin example
- Key management problem
- Key management antipatterns
- Key management solutions
- Discuss

UEFI

- Unified Extensible Firmware Interface
- Industry standard (Intel, AMD, Microsoft, etc.)
- Root of trust
- BIOS replacement
- Where is it? SPI flash

Factory keys

- X.509 public certs enrolled to UEFI firmware
- PK – Platform Key to update/delete KEK (OEM)
- KEK – Key Exchange Key to update/delete DB/DBX (OEM)
- DB – database of certs used to verify images (Microsoft)
- DBX – database of revoked certs (malware)

shim

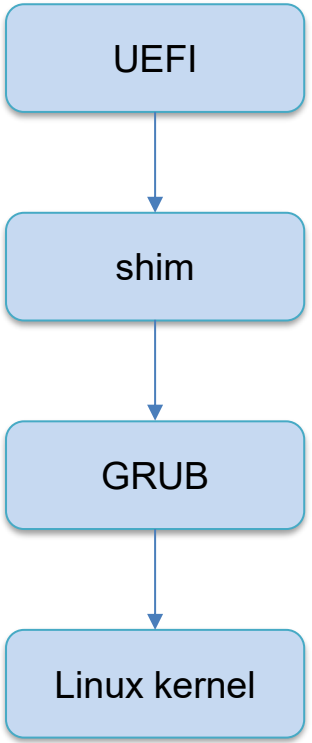
UEFI verifies executables against a database of public keys as the system boots.

shim is a software package designed to work as a first-stage booloader on UEFI systems.

shim embeds a distro-specific CA key as a trust root to verify subsequent programs.

shim is signed by Microsoft on behalf of trusted Linux distros like Debian so UEFI can boot Linux on x86 PCs using factory keys.

shim verifies GRUB and the Linux kernel against the distro key.



Signing shim

1. Fork rhboot/shim repo and build shim with your distro's certificate embedded inside.
2. Submit shim for signing to rhboot/shim-review repo.
3. Peer review
4. Microsoft review
5. Microsoft signs and returns shim

MOK

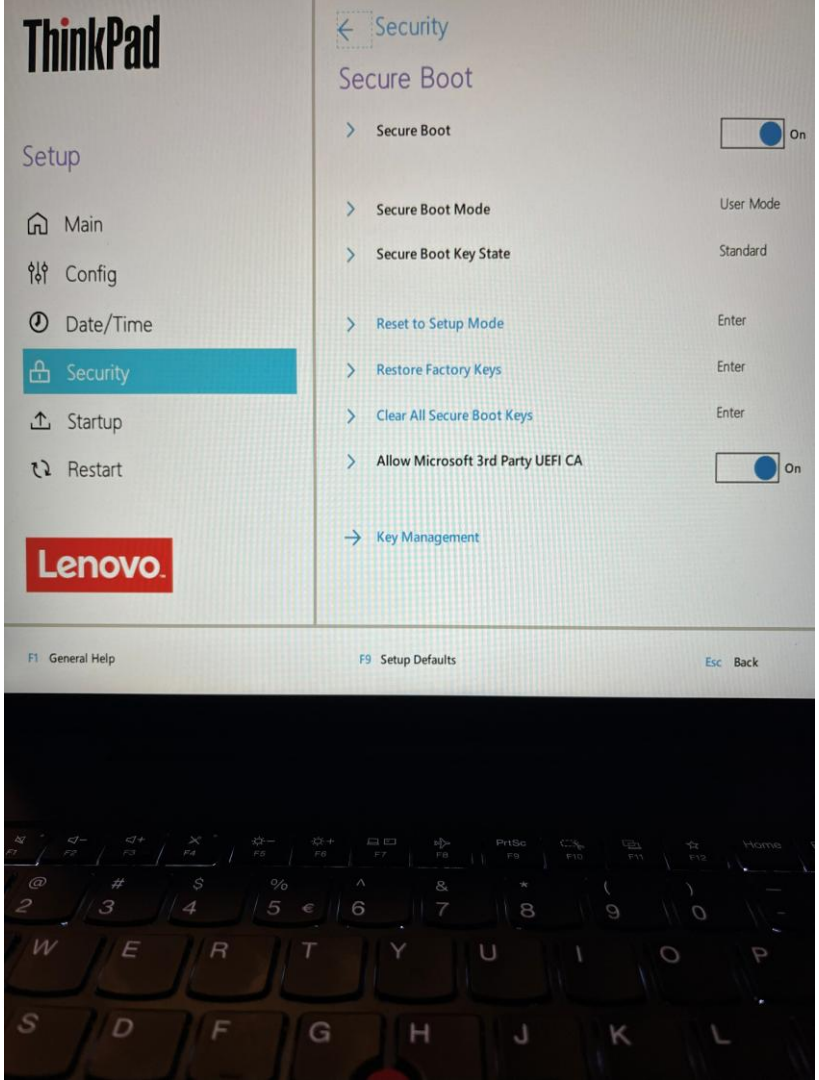
- MOK – Machine Owner Key (user-enrolled)
- Additional database of certs outside of factory DB/DBX
- UEFI NVRAM variables
- Introduced by shim
- Distro keys (in addition to distro key embedded in shim)
- User keys (self-signing)

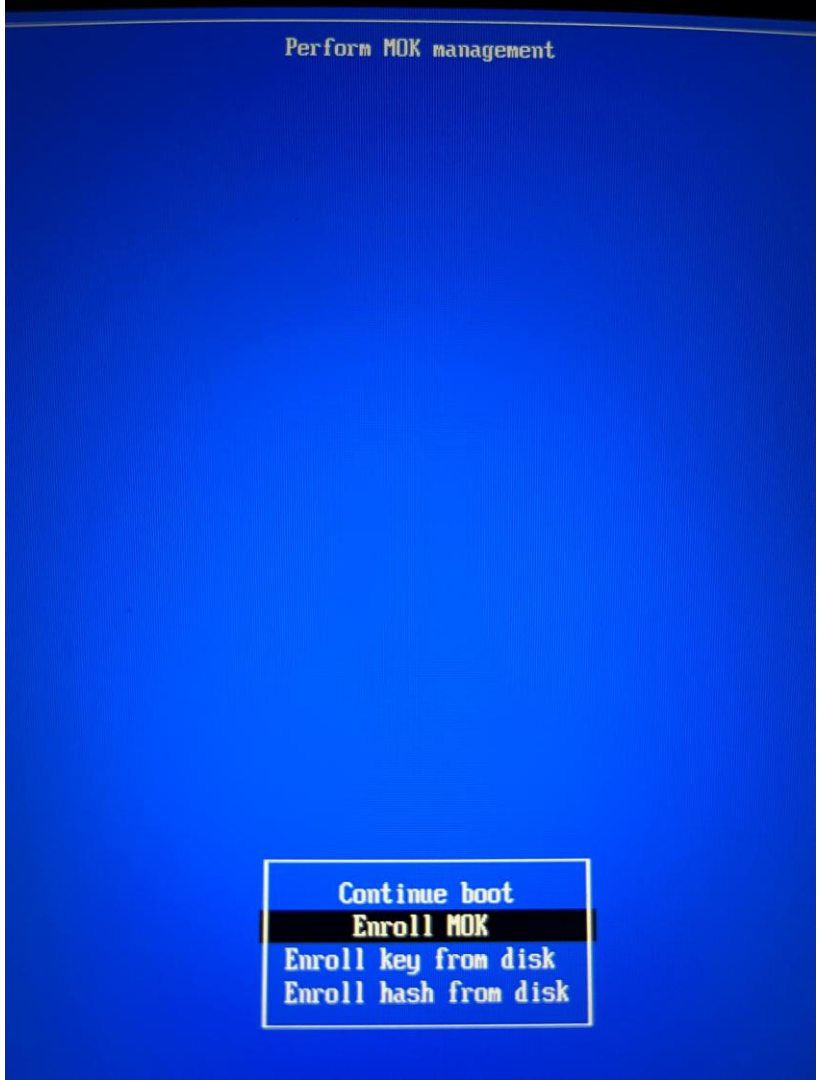
Bluefin

Universal Blue supports secure boot with our custom key

After the first installation, you will be prompted to enroll the secure boot key in the mokutil UEFI menu UI (QWERTY keyboard input and navigation).

Then, select **Enroll MOK**, and input **`universalblue`** as the password


ThinkPad
Setup
Main
Config
Date/Time
Security
Startup
Restart
Lenovo
Security
Secure Boot
Secure Boot
On
Secure Boot Mode
User Mode
Secure Boot Key State
Standard
Reset to Setup Mode
Enter
Restore Factory Keys
Enter
Clear All Secure Boot Keys
Enter
Allow Microsoft 3rd Party UEFI CA
On
Key Management
F1 General Help
F9 Setup Defaults
Esc Back


Perform MOK management
Continue boot
Enroll MOK
Enroll key from disk
Enroll hash from disk

Enroll MOK

```
❯ sudo mokutil --import path/to/public_key.der
```

When you run **`mokutil --import`**, it doesn't immediately modify the MOK database. Instead, it stages the request in a separate UEFI variable. On next reboot, **shim** detects the pending request and drops you into **MokManager**—a minimal pre-boot environment where you must physically confirm the enrollment by entering a password you set during the import. This physical presence requirement prevents remote attackers from enrolling malicious keys.

Fedora shim

```
❯ efibootmgr -v
BootCurrent: 0000
Timeout: 0 seconds
BootOrder:
0000,0010,0011,0012,0013,0014,0015,0018,0019,001A,001B,001C,001D, …
Boot0000* Fedora HD(1,GPT,b0eb66d0-1676-4fcb-aec8-
d1cebdf4c8bb,0x800,0x12c000)/\EFI\fedora\shimx64.efi
…
```

Universal Blue MOK

```
❯ mokutil --sb-state
SecureBoot enabled

❯ mokutil --list-enrolled | grep Subject:
    Subject: C=US, ST=Massachusetts, L=Cambridge, O=Red Hat, Inc.,
OU=Fedora Secure Boot CA 20200709, CN=fedoraca
    Subject: O=Universal Blue, OU=kernel signing, CN=ublue kernel
/emailAddress=security@universal-blue.org
```

Problem

- How do build systems sign build artifacts for UEFI Secure Boot without leaking the private key?
- Every Linux distro that supports UEFI Secure Boot grapples with key management.
- No reports of any known Linux distro's private key having been leaked.

Antipatterns

- Committing private key to version control
- GitHub, GitLab, and Bitbucket environment or repository variables
- File injection:

```
echo "${{ secrets.CERT }}" > /tmp/cert.pem
```

- GitLab API fetch:

```
curl --header "JOB-TOKEN: $CI_JOB_TOKEN" \
  "$CI_API_V4_URL/projects/$CI_PROJECT_ID/variables/MY_SECRET"
```

Short-lived tokens and credentials

- Jason Web Token (JWT)
- OpenID Connect (OIDC) access token
- AWS Identity and Access Management (IAM) credentials

- Hashicorp Vault
- OpenBao
- AWS Key Management Service (KMS)

Signing servers

- Unknown for buildd (Debian)
- pesignd for Koji (Red Hat, Fedora, CentOS)
- obs-signd for Open Build Service (SUSE, openSUSE, ParticleOS)
- madisongh/digsigserver for Yocto Project (OE4T)

Hardware Security Module

- Private keys should be kept in an HSM.
- HSM exposes a PKCS #11 library for rhboot/pesign to use.

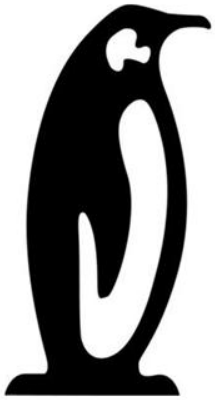
- Nitrokey HSM 2 ($65 USD one time)
- YubiHSM 2 ($650 USD one time)
- AWS CloudHSM ($1.45 USD an hour)

Discuss

- Is an HSM with a PKCS #11 library a prerequisite for secure signing?
- Is pesign what everyone does?
- Do we need so many signing servers?
- Is OBS the cross-distro build system solution?
- Do Red Hat or Debian-based distros have any interest in adopting OBS?
- Should obs-signd be integrated into other build systems?

https://www.linkedin.com/in/kraftwerk/





東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

